

Course: Graph Algorithms

Exercise #1: Graph centrality measures

Due given in the Moodle

Please note:

1. **Submission is in triplets not pairs. No single is allowed. (הגשה בשלושות בלבד)**
 2. The exercise must be submitted as a self-contained Jupyter Notebook file (.ipynb), that is, including everything required to run it. If any installation is required, it must be part of the code.
 3. The data files required to run must be attached and readable (pay attention to the path).
 4. A short report explaining the solution for each problem must be attached.
 5. Don't forget names and I.D.'s of the students.
-

In this assignment you will explore measures of centrality on a friendship network.
You can load the network using the library 'Networkx':

```
import networkx as nx
G1 = nx.read_gml('friendships.gml')
```

Question 1

Find the degree centrality, closeness centrality, and betweenness centrality of node 100.

This function should return a tuple of floats:

(degree_centrality, closeness_centrality, betweenness_centrality).

```
def answer_Q1():
    # YOUR CODE HERE
    raise NotImplementedError()
```

```
ans_Q1 = answer_Q1()
assert type(ans_Q1) == tuple, "You must return a tuple"
```

Question 2

Suppose you want to select one person in network G1 to send an invitation email. We expect that the person who receives the email will send it to their friends in the network. You want the email to reach as many nodes as possible. The email can be forwarded to multiple persons at the same time, but the travel distance of the email is limited to one

step, which means if the email travels more than one step in this network, it is discarded. Apply your knowledge in network centrality to select the best candidate for sending the email.

This function should return an integer, the chosen node.

```
def answer_Q2():  
    # YOUR CODE HERE  
    raise NotImplementedError()  
  
ans_Q2 = answer_Q2()
```

Question 3

Now the limit of the email's travel distance has been removed. Because the network is connected, regardless of who you pick, every node in the network will eventually receive the invitation. However, we now want to ensure that the email reaches nodes as quickly as possible (i.e., in the fewest number of hops). How will you change your selection strategy? Write a function to tell us who is the best candidate in the network under this condition.

This function should return an integer, the chosen node.

```
def answer_Q3():  
    # YOUR CODE HERE  
    raise NotImplementedError()  
  
ans_Q3 = answer_Q3()
```

Question 4

Assume the restriction on the email's travel distance is still removed, but now a malicious hacker has developed a strategy to remove a person from the network in order to disrupt the distribution of your invitation. He plans to remove people who act as bridges in the network. Identify the best possible person to be removed by the hacker?

This function should return an integer, the chosen node.

```
def answer_Q4():  
    # YOUR CODE HERE  
    raise NotImplementedError()  
  
ans_Q4 = answer_Q4()
```